

BodyFat Prediction Using Custom Gradient Descent Multiple Linear Regression On Bodyparts Measurements

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import math
from pathlib import Path
```

Lets get our dataset

```
In [2]: dataset_path = Path("../dataset/bodyfat.csv")
insurance_df = pd.read_csv(dataset_path)
insurance_df.head()
```

```
Out[2]:
```

	Density	BodyFat	Age	Weight	Height	Neck	Chest	Abdomen	Hip	Thigh	Knee	Ankle	Biceps	Forearm	Wrist
0	1.0708	12.3	23	154.25	67.75	36.2	93.1	85.2	94.5	59.0	37.3	21.9	32.0	27.4	17.1
1	1.0853	6.1	22	173.25	72.25	38.5	93.6	83.0	98.7	58.7	37.3	23.4	30.5	28.9	18.2
2	1.0414	25.3	22	154.00	66.25	34.0	95.8	87.9	99.2	59.6	38.9	24.0	28.8	25.2	16.6
3	1.0751	10.4	26	184.75	72.25	37.4	101.8	86.4	101.2	60.1	37.3	22.8	32.4	29.4	18.2
4	1.0340	28.7	24	184.25	71.25	34.4	97.3	100.0	101.9	63.2	42.2	24.0	32.2	27.7	17.7

Metadata

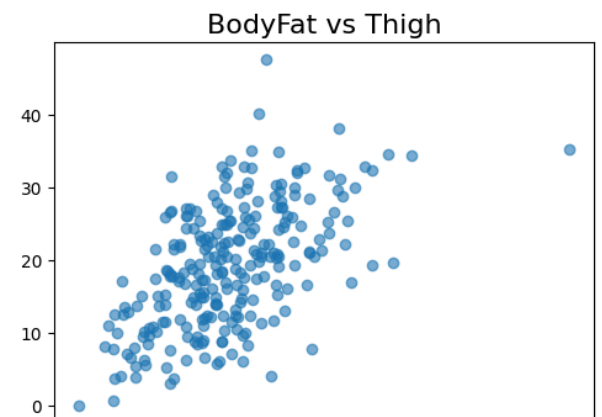
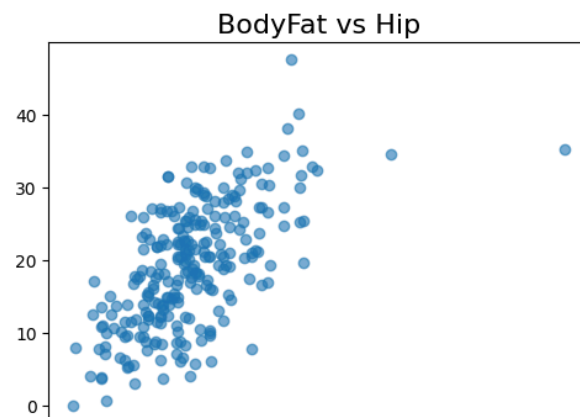
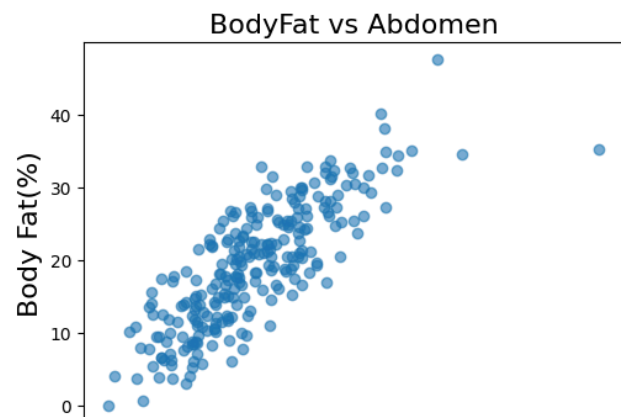
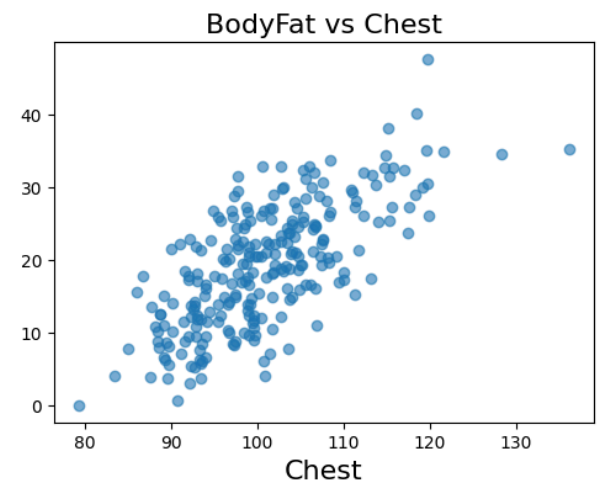
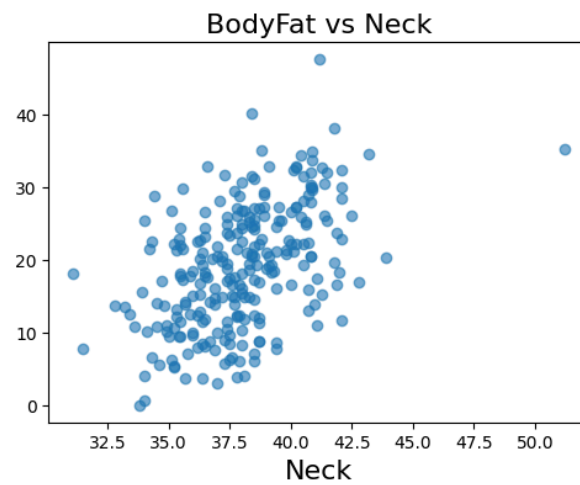
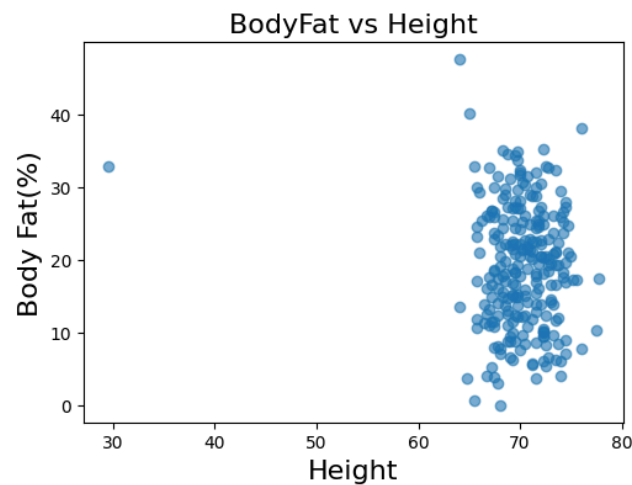
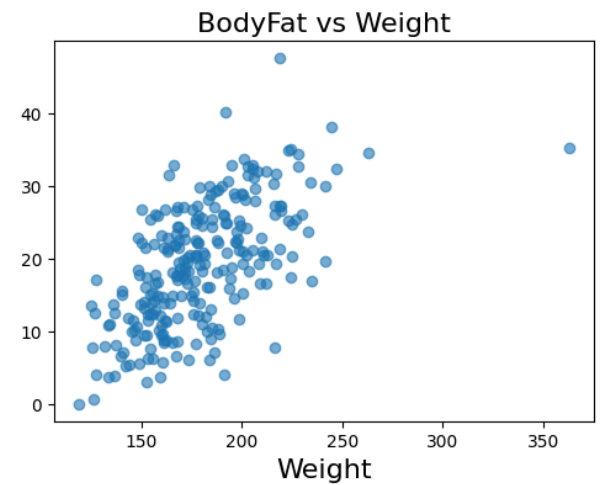
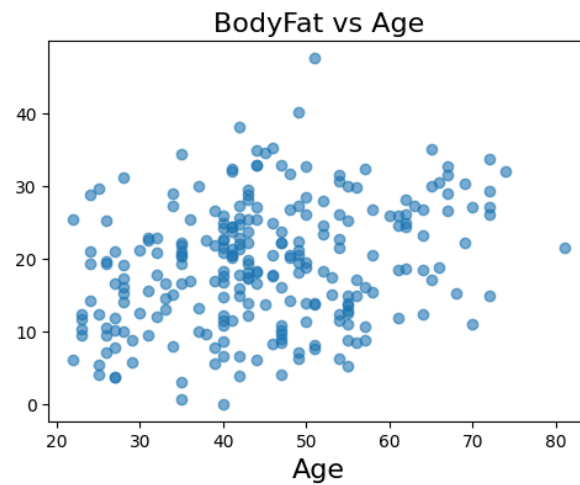
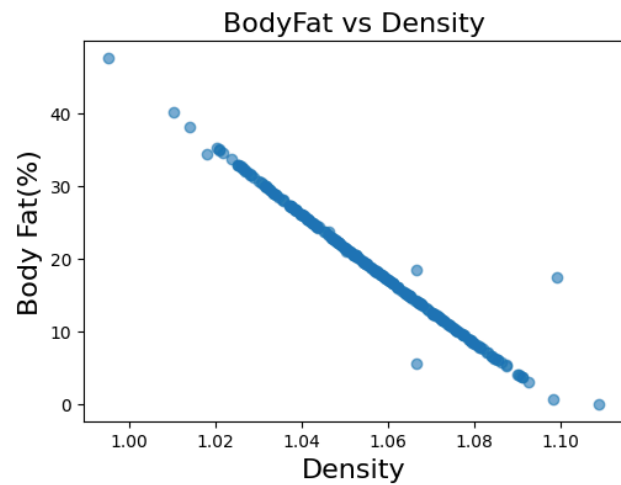
The variables listed above, from left to right, are:

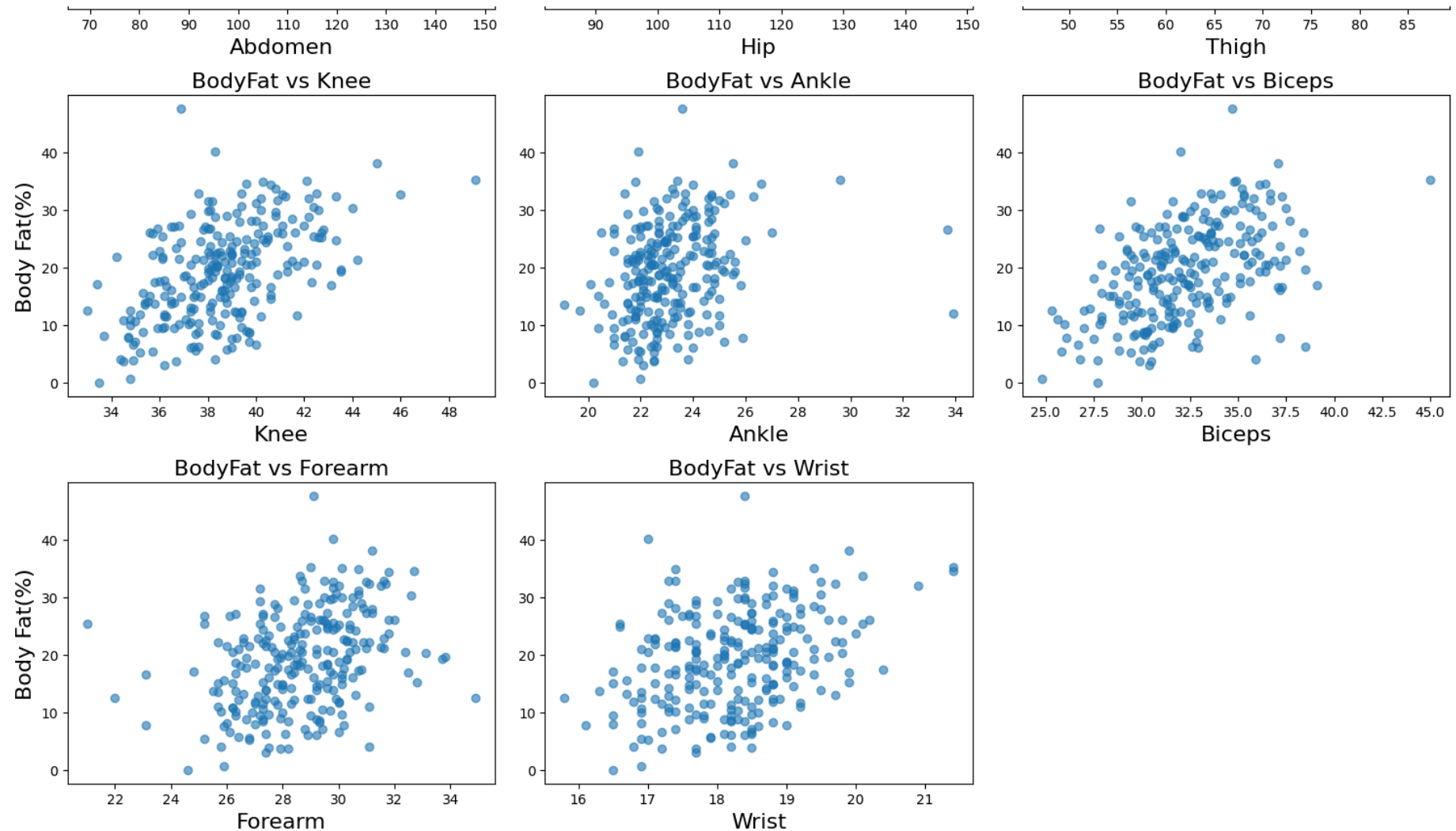
1. Density determined from underwater weighing
2. Percent body fat from Siri's (1956) equation -> **Our target variable**
3. Age (years)
4. Weight (lbs)

5. Height (inches)
6. Neck circumference (cm)
7. Chest circumference (cm)
8. Abdomen 2 circumference (cm)
9. Hip circumference (cm)
10. Thigh circumference (cm)
11. Knee circumference (cm)
12. Ankle circumference (cm)
13. Biceps (extended) circumference (cm)
14. Forearm circumference (cm)
15. Wrist circumference (cm)

Initial Data Visualization Analysis

```
In [3]: x_train = insurance_df.drop(columns = "BodyFat", inplace= False)
y_train = insurance_df["BodyFat"]
m, n = x_train.shape
headers = x_train.columns.to_list()
n_cols = 3
m_rows = math.ceil(n / n_cols)
fig, ax = plt.subplots(m_rows, n_cols, figsize = (n_cols * 5, m_rows * 4))
ax = ax.flatten()
for i in range(n):
    ax[i].scatter(x_train[headers[i]], y_train, alpha = 0.6)
    ax[i].set_xlabel(headers[i], fontsize = 16)
    ax[i].set_title(f"BodyFat vs {headers[i]}", fontsize = 16)
    if (i % 3 == 0):
        ax[i].set_ylabel("Body Fat(%)", fontsize = 16)
for j in range(n, len(ax)):
    fig.delaxes(ax[j])
plt.tight_layout()
plt.show()
```





We can see that almost all features (except Density) have positive linear relationship compared to bodyfat. The Height vs Bodyfat even though quite dense together because there seem to be an outlier person whose height is around 30 inches (76,2cm) which stretch the graph out. This is turned out to be a mistake typo from the scientists who conducted this dataset, so i will remove the error data entry.

However the bodyfat vs density graph form an almost perfect straight line, this is because the bodyfat in this dataset is directly computed using density. This means that if we try to run gradient descent on this dataset, then it would completely ignore the other features and would make their weights a very small number close to 0 and make the weight of density a very large one. Thus because we would want to predict

bodyfat using the other measurements (as not everyone has a hundred thousand of dollar equipment to measure their density), therefore we would want to drop the density column.

```
In [4]: insurance_df = insurance_df[insurance_df["Height"] > 40]
x_train = insurance_df.drop(columns = "BodyFat", inplace= False)
y_train = insurance_df["BodyFat"]
x_train.drop(columns= "Density", inplace = True)
headers = x_train.columns.to_list()
```

Feature Scaling

Next lets handle the data from other features as we can see the data range varies a lot from each features. We will use **Z-score normalization** for this dataset

```
In [5]: from sklearn import preprocessing

scaler = preprocessing.StandardScaler()
scaler.fit(x_train)
x_train_np = scaler.transform(x_train)
```

Custom Gradient Descent for Multiple Linear Regression

Initial data visualization analysis and feature scaling is done, now lets run our custom gradient descent using fully vectorization (If you haven't seen my custom batch gradient descent implementation, please take a look at the file src/bgd_regressor.py)

```
In [6]: import os
import sys

sys.path.append(os.path.abspath(os.path.join("../")))
```

```
In [7]: from src.bgd_regressor import BGDRegression #get our custom gradient descent class
import numpy as np

bgd_regress = BGDRegression()
alpha = 0.03
num_iteration = 10000
```

```

m, n = x_train_np.shape
w = np.zeros((n,))
b = 0.0
w, b, J_hist, w_b_hist = bgd_regress.run_bgd_regressor(x_train_np, y_train, w, b, alpha, num_iteration)

```

```

iteration 0: cost = 199.9653
iteration 1000: cost = 8.8980
iteration 2000: cost = 8.8001
iteration 3000: cost = 8.7903
iteration 4000: cost = 8.7879
iteration 5000: cost = 8.7871
iteration 6000: cost = 8.7869
iteration 7000: cost = 8.7868
iteration 8000: cost = 8.7867
iteration 9000: cost = 8.7867

```

As we can see the gradient hasn't move anywhere since iteration 8000, that is because the bgd_regressor has found the minimum for $J(w,b)$. The gradient descent has reached close to the minimum in just iteration 1000, this is thanks to feature scaling that our batch gradient descent ran very fast.

```

In [8]: #value of parameters w and b at minimum point of cost function J(w,b) = 8.7867
        for i in range(n):
            print(f"weight of {headers[i]}: w{i} = {w[i]:.4f}")
        print(f"bias b = {b:.4f}")

```

```

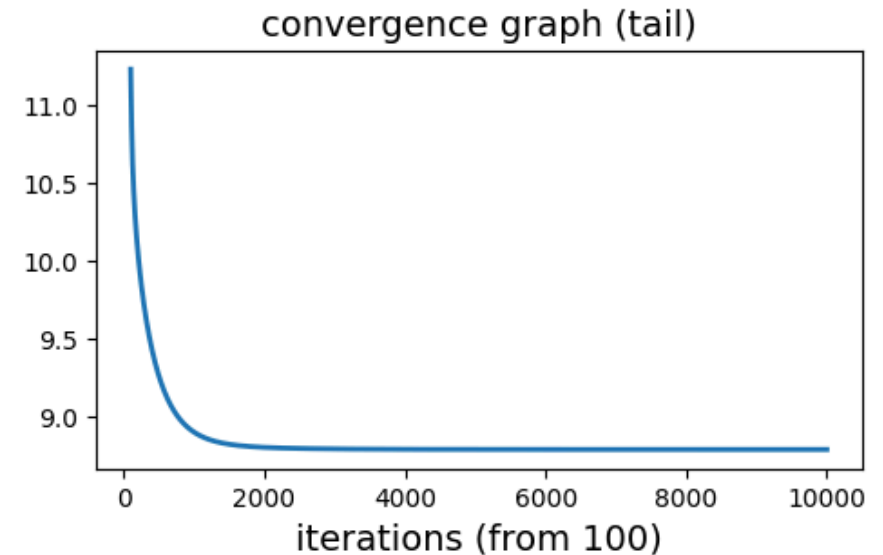
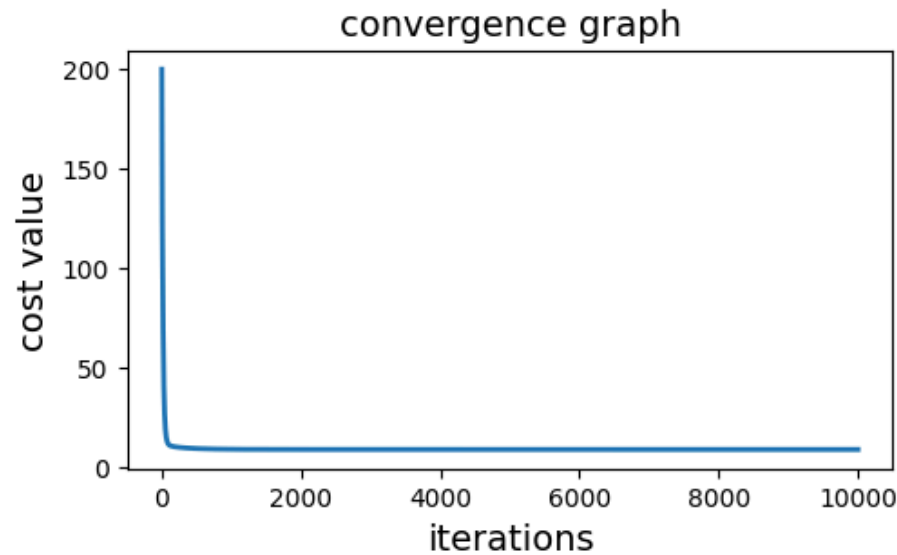
weight of Age: w0 = 0.7834
weight of Weight: w1 = -2.7350
weight of Height: w2 = -0.1182
weight of Neck: w3 = -1.1283
weight of Chest: w4 = -0.1641
weight of Abdomen: w5 = 10.3157
weight of Hip: w6 = -1.4603
weight of Thigh: w7 = 1.2493
weight of Knee: w8 = 0.0152
weight of Ankle: w9 = 0.3001
weight of Biceps: w10 = 0.5582
weight of Forearm: w11 = 0.9115
weight of Wrist: w12 = -1.5092
bias b = 19.0960

```

Looking at the result we can see that the weight for belly size ($w = 10.3157$) is massive compare to other features. This suggest that **abdomen size** is the **biggest predictor** of body fat percentage as each cm of Abdomen size multiply with the weight of 10 fold. Thigh ($w = 1.2493$) and forearm size ($w = 0.9115$) as well as age ($w = 0.7834$) does have some impact on bodyfat percentage compare to other features. What is interesting about the weight of **weight size** is that it is **negative**. This seem counter intuitive at first because larger weight should suggest that that person has more chance to have a higher body fat percentage. But the weight in multiple linear regression take all features as constant, and look at a single feature to figure its weight. A person with the same body part sizes, height, age but has a different weight. Where does that different weight comes from if everything else is the same? The answer is it comes from muscle. For example two person with the same chest size, say 30 cm. That two person has the same body parts size, same age, same height, but their weight is different. There can be only a explanation, muscle. As a cm of muscle weigh more than fat, and higher muscle suggest that the person has a the lower chance of carrying more bodyfat. The model discover this and assign negative weight ($w = -2.7350$) to that feature.

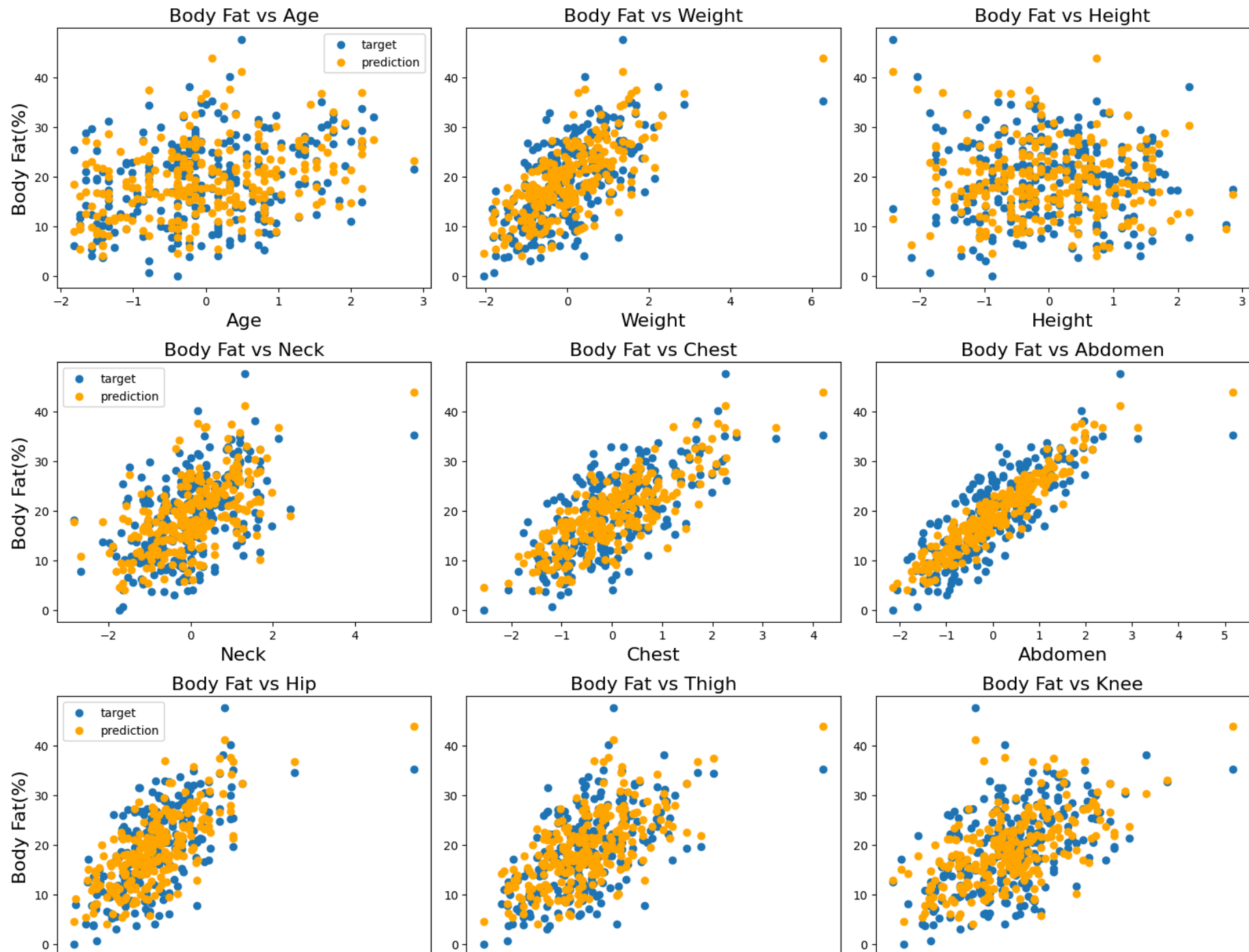
Convergence Graph

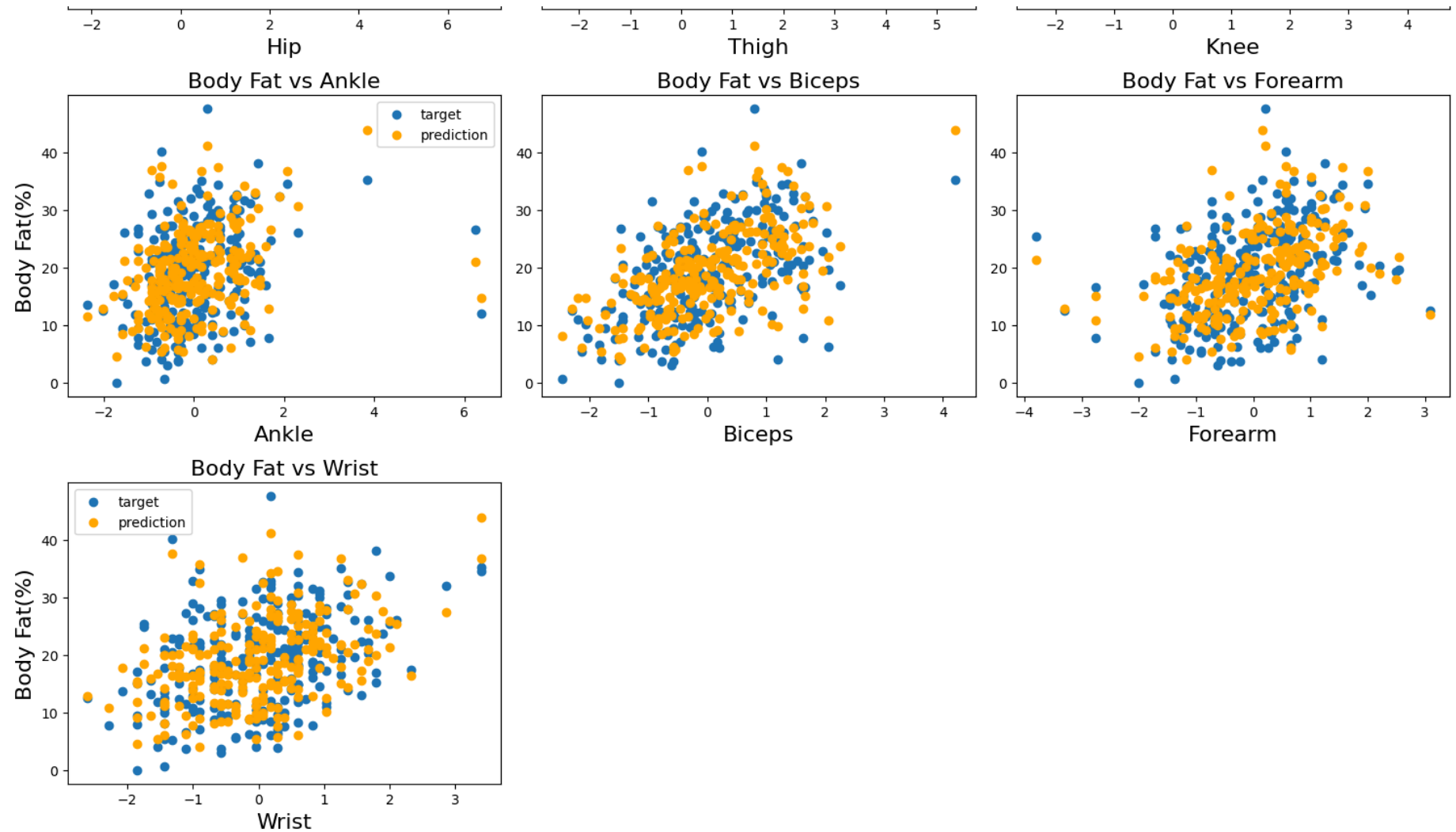
```
In [9]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize = (12,3))
ax1.plot(np.arange(len(J_hist)), J_hist, linewidth = 2.0)
ax1.set_xlabel("iterations", fontsize = 14)
ax1.set_ylabel("cost value", fontsize = 14)
ax1.set_title("convergence graph", fontsize = 14)
ax2.plot(100 + np.arange(len(J_hist[100:])), J_hist[100:], linewidth = 2.0)
ax2.set_xlabel("iterations (from 100)", fontsize = 14)
ax2.set_title("convergence graph (tail)", fontsize = 14)
plt.show()
```



```
In [10]: m, n = x_train.shape
         prediction = np.dot(x_train_np, w) + b
```

```
In [11]: n_cols = 3
         m_rows = math.ceil(n / 3)
         headers = x_train.columns.to_list()
         fig, ax = plt.subplots(m_rows, n_cols, figsize=(n_cols * 5, m_rows * 4))
         ax = ax.flatten()
         for i in range(n):
             ax[i].scatter(x_train_np[:, i], y_train, label = "target")
             ax[i].scatter(x_train_np[:, i], prediction, color = "orange", label = "prediction")
             ax[i].set_xlabel(headers[i], fontsize = 16)
             ax[i].set_title(f"Body Fat vs {headers[i]}", fontsize = 16)
             if (i % 3 == 0):
                 ax[i].set_ylabel("Body Fat(%)", fontsize = 16)
                 ax[i].legend()
         for j in range(n, len(ax)):
             fig.delaxes(ax[j])
         plt.tight_layout()
         plt.show()
```





Model Evaluation

```
In [13]: #rmse
rmse = math.sqrt(np.sum((prediction - y_train) ** 2) / m)
#mse
mae = np.sum(np.abs(prediction - y_train) / m)
#R square
```

```
sst = np.sum((y_train - np.mean(y_train)) ** 2)
ssr = np.sum((prediction - y_train) ** 2)
r_square = 1 - ssr / sst

print(f"RMSE = {rmse:.4f}")
print(f"MAE = {mae:.4f}")
print(f"R^2 = {r_square:.4f}")
```

RMSE = 4.1921

MAE = 3.4511

R^2 = 0.7463

Looks like our Root Mean Squared Error (RMSE) show that on average we are 4.1921% off, compare to Mean Absolute Error (MAE) is 3.4511%. This could be due to some outlier that the RMSE is a little bit away from MAE, but generally they are still quite low and show great result. Our R Square (R^2) score is quite good as it approximate 74.63%. This means that 74.63% of the variance is explained by our model on using the body parts measures. The remaining 25.37% could be potentially be explained if other measurement is added such as waist size or glute size are added and our predicted values will be closer to the target.